

- EasySplit — Documento de Entrega
 - Título do Projeto
 - Integrantes
 - 1. Descrição Geral do Produto e Funcionalidades
 - 2. Diagrama ER do Banco de Dados
 - 3. Detalhamento Técnico das Principais Partes do Código
 - 3.1 Application Factory — app.py
 - 3.2 Serviço de Divisão Igualitária — services/expense_split_service.py
 - 3.3 Serviço de Cálculo de Saldos — services/settlement_service.py
 - 3.4 Rotas de Grupos — routes/groups.py
 - 4. Arquitetura Adotada
 - 5. Tecnologias Utilizadas e Justificativa
 - 6. Como Rodar o Projeto
 - 7. Principais Desafios e Soluções

EasySplit — Documento de Entrega

Título do Projeto

EasySplit — Divisão de Despesas em Grupo

Integrantes

hugo lopes de almeida

caua caetano

igor maciel <!-- Insira os nomes dos integrantes aqui -->

1. Descrição Geral do Produto e Funcionalidades

O EasySplit é uma aplicação web para divisão igualitária de despesas entre grupos de pessoas. O sistema permite que usuários criem grupos, convidem amigos por e-mail, registrem despesas compartilhadas e visualizem de forma clara quem deve pagar a quem para quitar todas as dívidas.

Funcionalidades implementadas:

- Cadastro e autenticação de usuários (e-mail e senha com hash)
- Criação e gerenciamento de grupos de despesas
- Adição e remoção de participantes (com bloqueio quando há despesas vinculadas)
- Convite de usuários cadastrados por e-mail para entrar no grupo
- Registro de despesas com título, valor, data, pagador e divisão igualitária automática
- Dashboard financeiro com saldo por participante (a receber / a pagar / quitado)
- Sugestão automática de acertos (quem paga a quem e quanto)
- Registro de pagamentos entre participantes
- Chat interno por grupo
- Sistema de notificações in-app

2. Diagrama ER do Banco de Dados

```
users
  id (PK), nome, email (UNIQUE), telefone, password_hash,
  google_id, created_at, updated_at

groups
  id (PK), nome, owner_id (FK → users.id), created_at, updated_at

participants
  id (PK), nome, group_id (FK → groups.id), user_id (FK → users.id),
  created_at, updated_at
  UNIQUE(group_id, nome)

expenses
  id (PK), title, amount (NUMERIC 12,2), expense_date,
  group_id (FK → groups.id), payer_participant_id (FK → participants.id),
  created_at, updated_at

expense_participants
  id (PK), expense_id (FK → expenses.id),
  participant_id (FK → participants.id),
  divided_amount (NUMERIC 12,2), created_at, updated_at
```

```
UNIQUE(expense_id, participant_id)
```

```
group_invitations
```

```
id (PK), group_id (FK → groups.id),  
invited_user_id (FK → users.id),  
invited_by_user_id (FK → users.id),  
status (pending | accepted | declined),  
created_at, responded_at  
UNIQUE(group_id, invited_user_id)
```

```
payments
```

```
id (PK), group_id (FK → groups.id),  
payer_participant_id (FK → participants.id),  
receiver_participant_id (FK → participants.id),  
amount (NUMERIC 12,2), payment_date, created_at
```

```
messages
```

```
id (PK), group_id (FK → groups.id), user_id (FK → users.id),  
content, created_at
```

```
notifications
```

```
id (PK), user_id (FK → users.id), message, is_read, created_at
```

Relacionamentos:

- **users** → **groups** (1:N, dono do grupo)
- **groups** → **participants** (1:N)
- **users** → **participants** (1:N, participante vinculado a conta)
- **groups** → **expenses** (1:N)
- **participants** → **expenses** (N:1, pagador)
- **expenses** → **expense_participants** (1:N, divisão)
- **groups** → **group_invitations** (1:N)
- **groups** → **payments** (1:N)
- **groups** → **messages** (1:N)
- **users** → **notifications** (1:N)

3. Detalhamento Técnico das Principais Partes do Código

3.1 Application Factory — **app.py**

```

def create_app(config_overrides: dict | None = None) -> Flask:
    app = Flask(__name__)
    app.config.update(
        SECRET_KEY=os.environ.get("SECRET_KEY", "dev"),
        SQLALCHEMY_DATABASE_URI=f"sqlite:///db_path",
        SQLALCHEMY_TRACK_MODIFICATIONS=False,
    )
    db.init_app(app)
    login_manager.init_app(app)
    app.register_blueprint(auth_bp)
    app.register_blueprint(groups_bp)
    ...
    @app.errorhandler(404)
    def not_found(e):
        return render_template("errors/404.html"), 404

    @app.template_filter("brl")
    def brl(value) -> str:
        amount = Decimal(value or 0).quantize(Decimal("0.01"))
        return f"R$ {formatted}"

```

A função `create_app()` segue o padrão Application Factory, permitindo criar instâncias independentes da aplicação para testes e para produção. A `SECRET_KEY` é lida da variável de ambiente via `python-dotenv`, nunca hardcoded.

3.2 Serviço de Divisão Igualitária — `services/expense_split_service.py`

```

def split_amount_equally(amount: Decimal, participant_count: int) -> list[Decimal]:
    total_cents = int((to_money(amount) * 100).to_integral_value())
    base_cents = total_cents // participant_count
    remainder = total_cents % participant_count
    shares = []
    for index in range(participant_count):
        cents = base_cents + (1 if index < remainder else 0)
        shares.append(to_money(Decimal(cents) / 100))
    return shares

```

O algoritmo opera em centavos inteiros para evitar erros de arredondamento de ponto flutuante. O centavo restante (quando o valor não é divisível) é distribuído aos primeiros participantes da lista.

3.3 Serviço de Cálculo de Saldos — `services/settlement_service.py`

```
def calculate_group_summary(participants, expenses, payments=None):
    # Para cada participante:
    #   saldo = total_pago - total_devido
    # Sugestões: devedores pagam credores (algoritmo guloso)
    for balance in sorted_by_balance:
        while balance.balance < 0 and credores:
            credor = credores[0]
            valor = min(abs(balance.balance), credor.balance)
            settlements.append(SettlementSuggestion(
                payer=balance.participant,
                receiver=credor.participant,
                amount=valor
            ))
```

A fórmula de saldo é `saldo = total_pago - total_devido`. Saldo positivo significa "a receber"; negativo, "a pagar". O algoritmo de sugestão de acertos minimiza o número de transferências usando uma abordagem gulosa.

3.4 Rotas de Grupos — `routes/groups.py`

O blueprint `groups` centraliza todas as operações de grupos, participantes, despesas, pagamentos, convites e mensagens. A função auxiliar `_get_group_for_user_or_404()` garante que o usuário logado só acessa grupos nos quais é dono ou participante, retornando 404 em caso contrário — impedindo acesso não autorizado a dados de outros usuários.

4. Arquitetura Adotada

O projeto segue o padrão **Application Factory** com separação em camadas:

Camada	Pasta/Arquivo	Responsabilidade
Factory	<code>app.py</code>	Cria a app, registra blueprints, filtros, handlers de erro

Camada	Pasta/Arquivo	Responsabilidade
Extensões	<code>extensions.py</code>	Instâncias de <code>db</code> e <code>login_manager</code> sem importações circulares
Dados	<code>models/</code>	Classes SQLAlchemy mapeadas para tabelas SQLite
Validação	<code>forms/</code>	Formulários WTForms com validação server-side e proteção CSRF
Controle	<code>routes/</code>	Blueprints <code>auth</code> , <code>groups</code> , <code>notifications</code> , <code>oauth</code>
Negócio	<code>services/</code>	Lógica de divisão e cálculo de saldos, isolada e testável
Apresentação	<code>templates/</code>	Jinja2 com herança de <code>base.html</code>
Estilo	<code>static/css/</code>	CSS customizado

Esta separação permite testar a lógica de negócio (`services/`) de forma independente da camada HTTP, demonstrado em `tests/test_services.py`.

5. Tecnologias Utilizadas e Justificativa

Tecnologia	Versão	Justificativa
Python	3.14	Linguagem principal; ecossistema maduro para web
Flask	3.1.3	Microframework leve, ideal para MVPs e projetos acadêmicos
SQLAlchemy	2.0	ORM robusto com suporte a múltiplos bancos
Flask-Login	0.6.3	Gerenciamento de sessão com <code>@login_required</code>
Flask-WTF / WTForms	1.3 / 3.2	Formulários com validação server-side e proteção CSRF automática
Bootstrap	5 (CDN)	Framework CSS responsivo

Tecnologia	Versão	Justificativa
SQLite	—	Banco embarcado, sem necessidade de servidor externo
pytest / pytest-flask	9.0 / 1.3	Suite de testes com fixtures para Flask isolada
python-dotenv	1.2.2	Carregamento de variáveis de ambiente do arquivo <code>.env</code>
Jinja2	3.1.6	Motor de templates com herança e filtros customizados

6. Como Rodar o Projeto

```
# 1. Descompactar o projeto e entrar na pasta
cd easysplit

# 2. Criar e ativar o ambiente virtual
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Linux/macOS

# 3. Instalar dependências
pip install -r requirements.txt

# 4. Configurar variável de ambiente
copy .env.example .env
# Editar .env e alterar o valor de SECRET_KEY

# 5. Inicializar o banco de dados
flask --app app init-db

# 6. Iniciar o servidor de desenvolvimento
flask --app app run --debug

# 7. Acessar no navegador
# http://localhost:5000
```

Para rodar os testes:

```
.\venv\Scripts\pytest tests/ -v
```

7. Principais Desafios e Soluções

Desafio 1 — Arredondamento de centavos na divisão igualitária Ao dividir R 10,00 entre 3 pessoas, o resultado de R 3,333... não pode ser representado com 2 casas decimais sem erros acumulados. A solução foi converter o valor para centavos inteiros, fazer a divisão inteira e distribuir o centavo restante (módulo) entre os primeiros participantes. Isso garante que a soma das parcelas é sempre exatamente igual ao total original.

Desafio 2 — Prevenção de acesso não autorizado Era necessário garantir que um usuário não pudesse acessar grupos de outros usuários alterando o ID na URL. A solução foi a função `_get_group_for_user_or_404()`, que sempre filtra pelo `current_user.id` antes de retornar qualquer grupo, retornando 404 para acessos não autorizados.

Desafio 3 — Importações circulares com a Application Factory O padrão Flask com múltiplos blueprints e modelos SQLAlchemy frequentemente causa importações circulares. A solução foi centralizar as instâncias de `db` e `login_manager` em `extensions.py`, um módulo que não importa nada da aplicação, quebrando o ciclo.

Desafio 4 — Testes isolados com banco em memória Para os testes automatizados, era necessário um banco limpo a cada teste sem interferir no banco de desenvolvimento. A solução foi configurar `SQLALCHEMY_DATABASE_URI="sqlite:///memory:"` e `WTF_CSRF_ENABLED=False` nas fixtures do `conftest.py`, criando e destruindo todas as tabelas a cada sessão de teste.

Desafio 5 — Participante vinculado a conta de usuário Um participante pode ser criado manualmente (sem conta) ou ser um usuário registrado que aceitou um convite. Foi necessário manter o campo `user_id` nullable em `Participant` e implementar a lógica de vinculação ao aceitar o convite: se já existe um participante com o mesmo nome (sem conta), ele é vinculado ao usuário; caso contrário, um novo participante é criado.